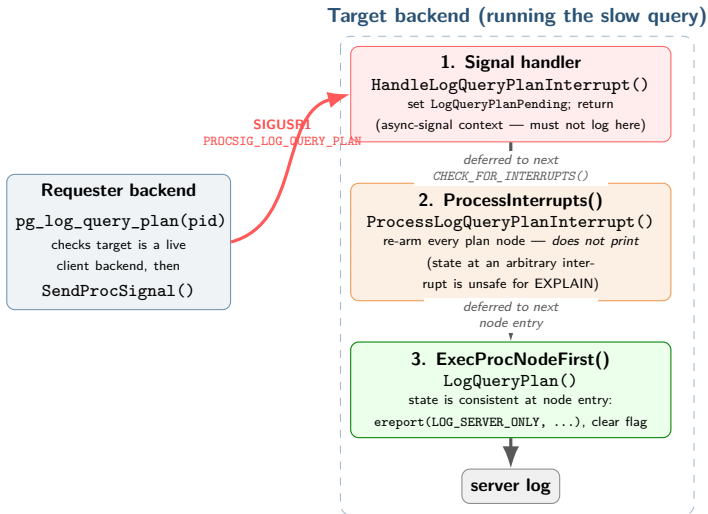


Architecture: from request to log line



Each hop defers the work to a point where more is safe to do:
signal-safe → **interrupt-safe** → **executor-safe**.

The safe-point trick: wrap, don't print

Re-arm the wrapper, don't print

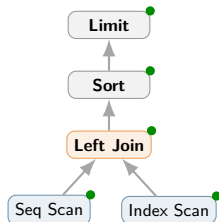
```
ProcessLogQueryPlanInterrupt()  
ExecSetExecProcNodeRecurse(ps):  
    node->ExecProcNode =  
        ExecProcNodeFirst // re-arm only  
  
ExecProcNodeFirst(node)  
    check_stack_depth();  
    if (LogQueryPlanPending) // NEW  
        LogQueryPlan(); // print,  
                        // clear flag  
    /* ... dispatch to real method ... */
```

Hop 2 swaps a function pointer — **no EXPLAIN** runs here.

The printing happens later, inside the wrapper, at node entry.

`ExecSetExecProcNodeRecurse` follows `lefttree`, `righttree`, `subPlan`, plus `Append` / `MergeAppend` / `BitmapAnd|Or` / `SubqueryScan` / `CteScan` / `CustomScan` — so whichever node the executor enters next prints the plan, then clears the flag. (A `WrapNodesInProgress` guard blocks re-entrant requests.)

Every node re-armed $\Rightarrow$ prints once



● re-armed node

How the EXPLAIN is built

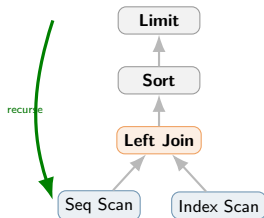
A string, built in a throwaway context

log_query_plan temp context

```
cxt = AllocSetContextCreate(...);
MemoryContextSwitchTo(cxt);
es = NewExplainState();
es->signaled = true; // analyze=false,
                    // costs=true
qd = GetCurrentQueryDesc();
ExplainStringAssemble(es, qd, ...);
ereport(LOG_SERVER_ONLY, es->str);
```

MemoryContextDelete(cxt)

It walks the tree that already exists



reads costs & rows;
never calls ExecProcNode()

Every EXPLAIN allocation lives in cxt and dies with it —

nothing leaks into the running query's per-query context.

No tuples pulled · private context · analyze=false ⇒ a plain cost EXPLAIN of the *live* plan tree, assembled into one string and logged.

...and why it can't touch the state

```
/* the only state-mutating call in the EXPLAIN path */  
if (planstate->instrument && !es->signaled)  
    InstrEndLoop(planstate->instrument);
```

Normal EXPLAIN ANALYZE

signaled = false

⇒ InstrEndLoop() **runs**

finalises the loop counters — *mutates* the instrumentation. Safe: the query is finished and ExecutorEnd has not cleaned up yet.

pg_log_query_plan()

signaled = true

⇒ InstrEndLoop() **skipped**

the query is mid-flight and still owns its Instrumentation; finalising it now would corrupt the counters the running query keeps updating.

With signaled *and* analyze=false (NewExplainState turns on only costs), the assembler never writes to executor-owned state — it only *reads* the existing PlanState tree.

explain.c:1878

Why the print is safe: three guarantees

The signal only *arms* the print; three independent properties make the print itself safe.

1. Safe point

Printed at *node entry* (`ExecProcNodeFirst`), where executor state is consistent — not at an arbitrary interrupt point.

2. Read-only walk

Traverses the already-built `PlanState` tree in a private memory context. No `ExecProcNode`, no tuples, no leaked allocations.

3. Gated write

The one mutating call, `InstrEndLoop`, is skipped under signaled; `analyze=false` $\Rightarrow$ no live counters read.

Residual, not eliminated: a backend stuck inside one long node logs only when it next advances; and an extension's `explain_per_plan_hook` still fires on the signalled path.